

Congress Trading Signal

La couche données

Extraction, identification, enrichissement et validation honnête des déclarations
boursières du Congrès américain — Chambre et Sénat, 2020–2026

Alice Lemaire · 28 juin 2026

Résumé

Les membres du Congrès américain déclarent leurs transactions boursières au titre du *STOCK Act*. Exploiter ces déclarations — par exemple via une stratégie de *copy-trading* — suppose au préalable une couche données propre, traçable et honnêtement validée. Ce rapport décrit la construction de cette couche, de l'acquisition des sources jusqu'à la table finale enrichie et validée, pour les **deux chambres** du Congrès sur sept ans (2020–2026). Le pipeline reconstruit **90 487 transactions** : **81 646** pour la **Chambre des représentants** (32 676 électroniques + 48 970 OCR) et **8 841** pour le **Sénat** (7 161 électroniques + 1 680 papier OCR). Chaque transaction est rattachée à son auteur (**99,99 %** des lignes à la Chambre, **100 %** au Sénat) et enrichie (ticker, secteur **GICS**→**ETF SPDR**, type d'actif, montant *midpoint*, ancienneté), produisant une table au **contrat de 12 champs métier**. L'extraction est validée contre **Quiver Quantitative** — vérité-terrain externe **jamais réinjectée** — au niveau transaction et par **scope** (électronique / OCR / les deux) : on retrouve **98,2 %** des trades Quiver sur l'électronique de la Chambre, **68,3 %** sur l'OCR et **86,2 %** sur l'ensemble ; **99,4 %** au Sénat. Quiver ne référence que les **actions cotées** : l'écart résiduel tient aux actifs **non cotés** (obligations municipales, fonds privés), hors de son périmètre, et à la **lecture OCR des dates** sur les formulaires manuscrits — seule véritable limite d'extraction. La correction est figée par un **golden octet-à-octet** (**125** fichiers à la Chambre, **76** au Sénat) et des reproductions fonction par fonction.

1 Introduction & contexte

Depuis le **STOCK Act** (2012), les membres du Congrès américain doivent divulguer publiquement leurs transactions sur titres dans un *Periodic Transaction Report* (PTR), en principe sous **45 jours**. L'intuition de recherche — documentée académiquement (Ziobrowski *et al.* 2004/2011, Karadas 2019) — est que ces personnes, par leur appartenance à des commissions clés (Finance, *Armed Services*, Intelligence) ou leur exposition à l'information, produisent par leurs déclarations un **signal potentiellement exploitable**. Une stratégie de *copy-trading* consiste à répliquer ces transactions *après* leur publication (*anti-look-ahead* : on date chaque transaction par sa date de divulgation, jamais par une date postérieure connue).

Avant toute stratégie, il faut une **couche données** irréprochable : extraire fidèlement les déclarations des deux chambres, les rattacher à la bonne personne, les enrichir (ticker, secteur, montant) et les valider honnêtement. C'est l'objet de ce rapport. La stratégie et le backtest associés constituent un travail de recherche séparé, hors du périmètre traité ici.

2 Sources & acquisition

Le pipeline part de **sources publiques officielles**, complétées d'un **référentiel d'élus** et de **deux services externes** (un modèle de vision pour les scans, un agrégateur pour la validation). On indique ici où l'on récupère quoi ; le *traitement* de chaque source est détaillé au §4.

- **Chambre des représentants** — *House Clerk*. Source publique sans barrière : une archive ZIP annuelle contenant un **index XML** (`{an}FD.xml`) qui liste tous les dépôts. On retient les PTR (`FilingType=P`) et on télécharge le PDF de chacun.

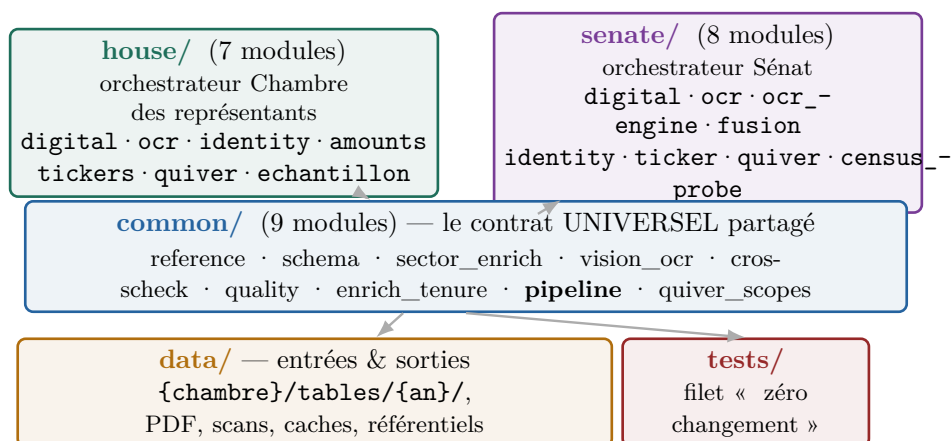
- **Sénat** — *eFD* (*Electronic Financial Disclosure*). L'API de recherche du Sénat (`report_types=[11]` = PTR), accessible après acceptation d'un formulaire d'accord. Chaque dépôt est soit une **page HTML** électronique, soit une **image papier** scannée.
- **Référentiel des élus** — *congress-legislators*. Une déclaration ne donne qu'un *nom* ; ce jeu de données ouvert fournit, pour chaque membre, son identifiant officiel (`bioguide_id`), son **parti** et ses **commissions** — avec un repli sur des YAML embarqués dans le dépôt pour rester reproductible hors-ligne.
- **Lecture des scans** — *Claude Vision*. La moitié des déclarations de la Chambre et tout le papier du Sénat n'existent que comme **images**. On les lit avec l'API Claude Vision (`claude-sonnet-4-6`), qui *lit et structure* directement le formulaire en transactions (sortie JSON stricte, cache versionné).
- **Validation** — *Quiver Quantitative*. Agrégateur commercial des transactions du Congrès, utilisé comme **vérité-terrain externe** : on mesure si notre extraction concorde, **sans jamais réinjecter** ses données.

Deux dates par déclaration. Chaque transaction porte une **date d'opération** (`transaction_date`, quand le trade a eu lieu) et une **date de divulgation** (`disclosure_date`, quand le dépôt devient public). Toute la chronologie s'appuie sur la divulgation — principe **anti-look-ahead** : une stratégie ne peut copier un trade qu'*après* sa publication, jamais à sa date d'opération, antérieure et inconnue du marché au moment du dépôt.

3 Architecture & méthodologie

3.1 Un contrat partagé + deux orchestrateurs jumeaux + données + tests

Le code suit une **architecture symétrique** : un **contrat universel partagé** (`common/`, 9 modules) et **deux orchestrateurs jumeaux** (`house/` et `senate/`) qui s'appuient dessus. `common/` porte ce qui est commun aux deux chambres — schéma, référentiel, OCR de référence, secteur, validation, qualité, orchestration ; ce qui dépend de la source — matcher d'identité, montants, ticker, OCR de production, réconciliation Quiver — vit dans `house/` et `senate/`. Les données et les tests complètent l'ensemble.

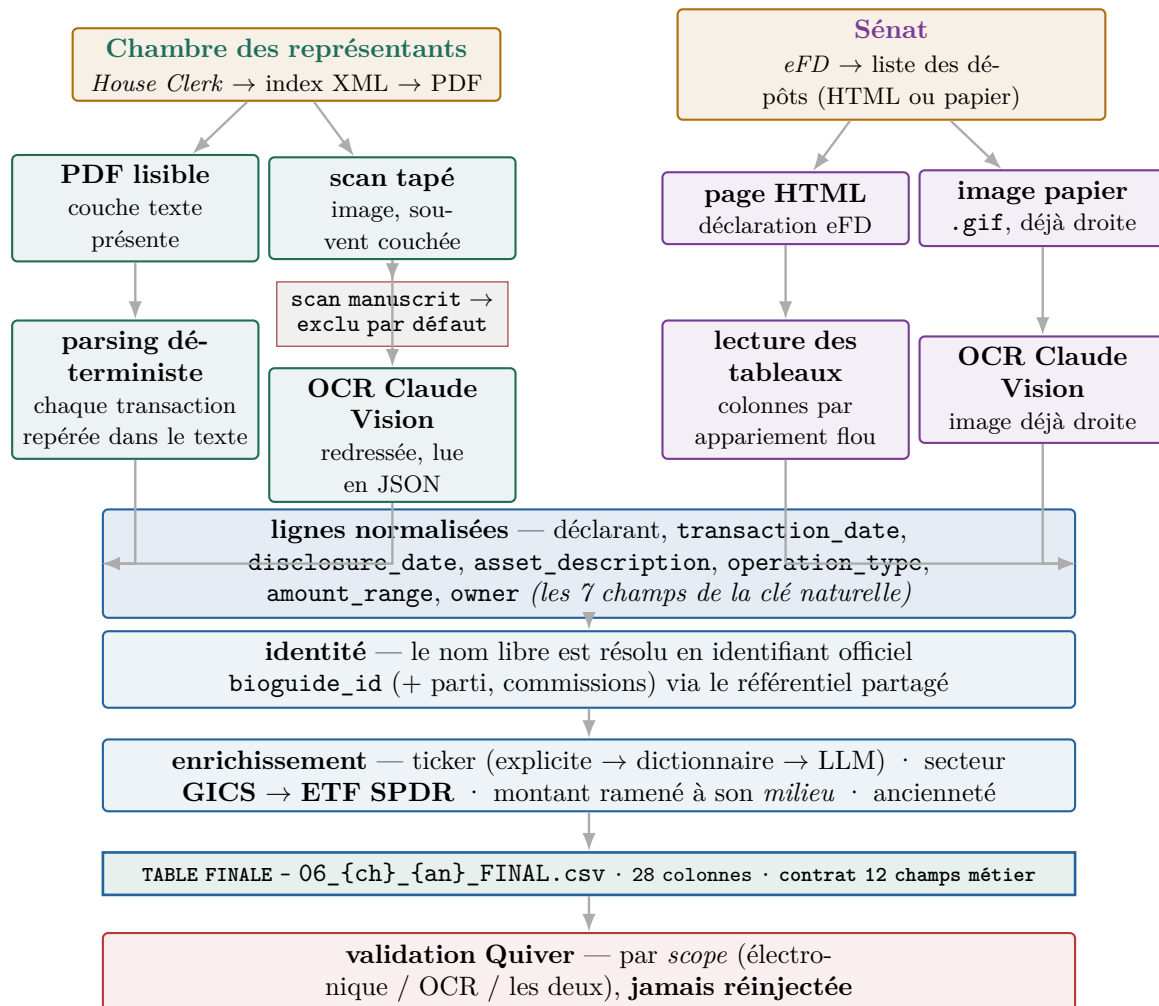


Les deux orchestrateurs portent les **mêmes rôles** (`digital`, `ocr`, `identity`, `quiver`...); seules leurs implémentations diffèrent, là où les sources diffèrent. L'unique asymétrie de structure est `senate/fusion.py` (§4.6) : côté Chambre des représentants, la fusion électronique+OCR vit dans `house/ocr.py`.

3.2 Le flux de bout en bout (le schéma à lire en premier)

Le schéma ci-dessous se lit **en premier** : il montre, de bout en bout, comment une déclaration devient une ligne de la table finale. En haut, les **deux sources** (Chambre des représentants *vs* Sénat); au milieu, la **fourche électronique / scanné** qui oriente chaque document vers son extraction ; en bas,

le **tronc commun** partagé par les deux chambres — identité, enrichissement, table finale, validation. L'orchestration est tenue par `common/pipeline.py` (`build_steps`) : **6 étapes** en sous-processus, suivies d'un **rapport qualité** hors-ligne, en lecture seule (`common/quality.py`).



De haut en bas : acquisition (deux sources) · tri puis extraction (quatre pistes, une par forme de document) · tronc commun partagé (identité → enrichissement → table finale → validation).

3.3 À quoi ressemble la donnée

La couche finale, ce sont **14 fichiers** `06_{chambre}_{an}_FINAL.csv` (7 ans × 2 chambres), rangés sous `data/house/tables/{an}/` et `data/senate/tables/{an}/`. Chaque **ligne** est une transaction et porte **28 colonnes** (détail en annexe 11.3). Concrètement :

Une transaction cotée (Sénat). Le sénateur M000355 — *A. Mitchell McConnell, Jr.*, Républicain, KY — déclare l'achat de **WFC** (*Wells Fargo & Company*), une action (Stock) du secteur **Financials** mappé sur l'ETF **XLF**, pour **\$1 001–15 000** (milieu **8 000,5 \$**), détenue par le conjoint, date jugée *plausible*. Tous les champs « métier » sont renseignés.

Une transaction non cotée (Chambre des représentants), en contraste, montre les **trous légitimes** : le représentant *Jefferson Shreve* déclare une *JP Morgan 3-year auto callable buffer note* — un produit structuré qui n'a **ni ticker, ni secteur, ni ETF** (ces colonnes sont vides) parce qu'il n'est *pas* coté en Bourse. Ce n'est pas un défaut d'extraction : il n'existe pas de symbole à lui attribuer (§4.7).

Autour de ces tables, des fichiers intermédiaires préfixés par **rôle** — 03_ index, 05_ échecs de parsing, 06_ électronique, 06b_ OCR, 07*_ réconciliation Quiver — tracent chaque étape et restent inspectables.

4 Construction de la donnée, étape par étape

Cette section déroule le **flux de bout en bout** (§3.2) dans l'ordre où le pipeline construit une transaction : d'abord l'identité du déclarant, puis le tri des formats, l'extraction de chaque piste (électronique *et* scannée, pour les deux chambres), enfin l'enrichissement et la table finale. Chaque étape en donne le **résultat chiffré**.

4.1 D'abord, savoir qui a déclaré : la résolution d'identité

Une déclaration n'identifie son auteur que par un *nom libre*, dont la graphie varie d'un dépôt à l'autre — « Hon. Earl L. Carter », « Buddy Carter », « N. Pelosi ». Sans clé stable, un même élu serait compté plusieurs fois et ne pourrait être rattaché ni à son parti ni à ses commissions. La résolution d'identité ramène donc chaque nom à l'**identifiant officiel** du membre, le `bioguide_id` attribué par le *Biographical Directory* du Congrès (par ex. P000197 pour Nancy Pelosi) : unique, il rend chaque ligne rattachable, déduplicable et joignable au référentiel des élus.

Ce référentiel est partagé par les deux chambres. `common/reference.py` charge les législateurs et leurs commissions (jeu *congress-legislators*, avec repli sur des YAML embarqués) et en dérive, pour chaque `bioguide_id`, le parti, les commissions et un drapeau « commission clé », ainsi qu'un **index de noms tolérant** qui enregistre plusieurs graphies par élu — nom de famille (y compris le dernier mot d'un nom composé), prénom, second prénom et surnom officiel. Chaque chambre construit ensuite son propre **matcher**, car les pièges diffèrent selon la source.

À la Chambre des représentants (`house.identity`), la résolution d'un couple (nom, prénom) suit une cascade à trois niveaux : un **override manuel**, réservé à un unique cas irréductible (« Nicholas V. Taylor », c'est-à-dire Van Taylor) ; à défaut, une **correspondance exacte** dans l'index, après normalisation des accents, des titres (*Hon.*, *Dr.*) et des suffixes, et en essayant les diminutifs usuels (*William* → *Bill*) ; en dernier recours, un **repli sur le nom de famille** lorsqu'il ne désigne qu'un seul élu. Au Sénat (`senate.identity`), où les homonymes sont plus fréquents, la cascade ajoute une **désambiguïsation** : à nom de famille égal, on privilégie d'abord un siège au Sénat, puis le titulaire *en exercice*, enfin une concordance de prénom ou d'initiale. Chaque chambre ne fige qu'un seul override (Van Taylor à la Chambre, J.D. Vance au Sénat). L'identité résolue, `house/digital.py:join_identity` et `senate/identity.py:enrich` y joignent le parti, les commissions et le drapeau de commission clé issus du référentiel partagé.

La résolution rattache **99,99 %** des lignes de la Chambre (81 642 / 81 646) et **100 %** de celles du Sénat. Les quatre lignes restantes forment une *seule* déclaration de 2023 (Ada Norah Henriquez, Porto Rico) que le matcher ne rattache à aucun `bioguide_id`. À la Chambre, **256** identifiants couvrent **275** graphies de noms : l'écart correspond exactement aux variantes orthographiques d'un même élu ramenées à un identifiant unique — ce que la résolution corrige.

4.2 Ensuite, trier les formats : électronique ou scanné ?

Une même déclaration peut arriver sous deux formes qui n'appellent pas le même traitement : un *texte* directement exploitable, que l'on peut analyser ligne à ligne, ou une *image* qu'il faut d'abord lire par OCR. L'orientation n'est pas neutre — router un scan vers l'analyse de texte perd des transactions, router un PDF déjà lisible vers la vision gaspille des appels d'API — et c'est elle qui commande, en amont, toute la suite du pipeline.

À la Chambre des représentants, le format n'est pas annoncé : il faut le déterminer. `house/digital.py:build_manifest` ouvre chaque PDF avec `pdfplumber` et applique un test binaire, la présence d'une couche de texte (`bool(text.strip())`) : un texte non vide classe le document comme *lisible*, un texte vide comme *non lisible* — donc destiné à l'OCR —, un fichier manquant comme *absent*. Le verdict, consigné dans `04_download_manifest.csv`, oriente tout le reste, et sa simplicité fait sa robustesse. Au Sénat, le tri n'a pas à être calculé : la recherche eFD le fournit déjà, chaque dépôt portant son type — `ptr` pour une déclaration HTML électronique, `paper` pour une image scannée.

Ce tri d'apparence anodine est la **fourche** du pipeline : il fixe la composition finale du corpus —

32 676 transactions électroniques et 48 970 issues de l'OCR à la Chambre, 7 161 et 1 680 au Sénat — et dirige chaque document vers la piste d'extraction décrite dans les sous-sections suivantes.

4.3 La piste électronique de la Chambre (PDF lisibles)

Sur un PDF lisible du Clerk, les transactions ne se présentent pas comme un tableau régulier : une même opération peut s'étaler sur plusieurs lignes, son montant déborder sur la suivante, le ticker se glisser entre parenthèses et le libellé de l'actif courir sur des lignes de continuation. Un simple découpage ligne à ligne en raterait une bonne part ; l'extraction doit donc reconstruire chaque transaction avant de la lire.

`house/digital.py:run_year` enchaîne les étapes. `load_ptr_index` lit l'index XML de l'année et en tire la liste des dépôts (`03_ptr_index.csv`). Le cœur du travail revient à `parse_ptr`, en deux temps : il **recolle** d'abord les lignes coupées — un montant scindé et sa continuation sont réassemblés en une seule ligne (`_join_split_lines`) — puis repère chaque transaction par une expression régulière (`TXN_RE`) qui en fixe la signature : un code d'opération (achat P, vente S, échange E), deux dates et une fourchette de montant. La transaction localisée, il **remonte de quelques lignes** pour rattacher la description de l'actif et le ticker, que le formulaire imprime au-dessus ou autour de la ligne d'opération. `join_identity` y joint ensuite l'identité et les métadonnées du dépôt, et `finalize` calcule la clé naturelle, numérote les lots répétés (`occurrence_index`) et déduplique (§4.8).

Le résultat est écrit dans `06_house_{an}_transactions.csv`. Le contrôle est explicite : tout PDF lisible ne rendant aucune transaction est consigné dans `05_parse_failures.csv` — aucun échec n'est silencieux —, et le « taux de parsing » mesure la part des PDF lisibles ayant produit au moins une ligne. Au total, la piste électronique de la Chambre fournit **32 676** transactions, pour un taux de parsing de **99–100 %**.

Dans la source. Un PTR électronique de la Chambre se lit comme un tableau propre (cf. annexe 11.1) :
« Amazon.com, Inc. — Common Stock (**AMZN**) [ST] · Purchase · 03/16/2018 · \$1 001–\$15 000 ». Le ticker est entre parenthèses, le type d'actif entre crochets — d'où le parsing déterministe.

4.4 La piste électronique du Sénat (HTML eFD)

Le Sénat ne publie pas de PDF mais des pages HTML, derrière un garde-fou CSRF : avant toute requête, `senate/digital.py:accept_agreement` accepte le formulaire d'accord de l'eFD (jeton `csrfmiddlewaretoken` et en-tête `X-CSRFToken`). La difficulté d'extraction tient ensuite aux tableaux eux-mêmes, dont l'intitulé et l'ordre des colonnes varient d'une déclaration à l'autre.

La séquence est portée par `run_year`, qui interroge la liste des dépôts (`fetch_ptr_list`, `report_types=[11]`) puis récupère chaque déclaration (`fetch_report`, avec mise en cache sur disque — un re-run ne retélécharge rien). `parse_electronic` lit alors les tableaux avec `pd.read_html`, qui renvoie des *DataFrames* bruts ; c'est `_find_col` qui y retrouve les bonnes colonnes par **appariement flou** — on retient la colonne dont l'intitulé contient les bons mots-clés (« ticker », ou « asset » et « name ») —, seule approche robuste face à des en-têtes irréguliers. L'identité, la résolution du ticker et la déduplication sont ensuite déléguées à `senate/identity.py:enrich`, qui applique le matcher du Sénat (§4.1) et la même clé naturelle qu'à la Chambre. La sortie est `06_senate_{an}_transactions.csv`.

Cette piste fournit **7 161** transactions électroniques pour le Sénat.

Dans la source. Une déclaration eFD du Sénat est une page web (cf. annexe 11.1) : « **YUM** · Yum! Brands · Sale (Full) · \$1 001–\$15 000 · Joint », avec le ticker cliquable et la date de transaction — d'où la lecture par `pd.read_html`.

4.5 La piste scannée de la Chambre (OCR Claude Vision)

L'autre moitié des déclarations de la Chambre n'existe que sous forme d'images. Un recensement des 547 PDF scannés (`_scan_census_547.csv`) les répartit en trois familles : 74 documents tapés et droits (cluster A), 322 tapés mais dont l'image est *couchée* (cluster B, le gros du volume) et 151 manuscrits (cluster C). Un modèle de vision lit mal une page tournée, et plus mal encore une date écrite à la main — deux difficultés traitées séparément.

La première est l'orientation, à corriger avant toute lecture. Demander directement l'angle de rotation au modèle se révèle peu fiable (il répond presque toujours « 90 ») ; on procède donc par **reconnaissance** plutôt que par estimation : `detect_rotation` lui présente la page sous les quatre orientations (`ROT_CANDIDATES = [0, 90, 180, 270]`) et lui fait désigner celle qui est droite. Ce redressement fait progresser nettement la concordance OCR (d'environ 60 % à 65 %) ; le reliquat d'erreur tient à l'ambiguïté *intrinsèque* des dates manuscrites, non à la rotation.

L'extraction proprement dite, dans `house/ocr.py:run_ocr_year`, force un appel d'outil (`tool_use` sur `record_transactions`, modèle `claude-sonnet-4-6`, sept tentatives avec *backoff*) : le modèle répond dans un schéma JSON strict, sans texte libre à re-parser, et chaque résultat est mis en cache, versionné par `prompt_sha` — un re-run ne re-paie que les lots en erreur. Le prompt avait été durci en amont sur un échantillon stratifié des trois clusters (`house/echantillon.py`), pour mesurer la qualité avant le passage à l'échelle. Le ticker, absent des formulaires papier, est ré-associé à partir des symboles vus en électronique, puis complété par une passe LLM. Le volume étant considérable, la fusion électronique + OCR, l'enrichissement et la validation Quiver se font **année par année** pendant le run, qui écrit directement `06_house_{an}_FINAL.csv`.

Le cluster C manuscrit est, lui, **exclu par défaut** : ses dates sont trop incertaines pour une stratégie qui les date, et une sonde comparant Opus à Sonnet sur ces scans a confirmé que le goulot n'est pas le modèle mais la lisibilité de l'écriture, pour un coût sensiblement plus élevé. Le code le conserve comme catégorie tracée sans l'exécuter, hormis une récupération ciblée des plus gros déposants. Au total, la piste scannée fournit **48 970** lignes, à la concentration extrême : Rohit Khanna en représente à lui seul **62,9 %**, et le trio de tête (Khanna, McCaul, Harshbarger) **92,4 %** — de gros déposants qui déclarent exclusivement sur papier.

Dans la source. Un scan de la Chambre est un **formulaire à cases** (cf. annexe 11.1), souvent *couché* : une transaction = une ligne où l'on coche le type (Purchase/Sale), la date, et la **fourchette de montant** (une colonne à cocher) — d'où le besoin de deskew puis de Vision.

4.6 La piste papier du Sénat — et pourquoi la fusion y est séparée

Au Sénat, le papier est marginal — cinq sénateurs seulement — mais bien réel : des images `.gif`, cette fois *droites*, qui ne demandent pas de redressement. `senate/ocr.py` les lit avec le même moteur Vision (figé dans `senate/ocr_engine.py`) et écrit la table OCR 06b. La difficulté est ailleurs : avec si peu de données, un dictionnaire de tickers reconstruit année par année serait trop pauvre.

C'est ce qui justifie la **seule véritable asymétrie d'architecture** entre les deux chambres. Là où la Chambre, riche en volume, fusionne et enrichit *pendant* chaque année (§4.5), le Sénat confie ces étapes à un module dédié, `senate/fusion.py`, exécuté une fois toutes les années assemblées. Sa fonction `main` procède en deux temps : d'abord une fusion non destructrice *par année* (électronique + OCR, dédoublée sur le couple (`natural_key_hash`, `occurrence_index`)) ; puis un enrichissement sur le **corpus complet, en une seule passe** — `senate/ticker.py:resolve_tickers` puis `common/sector_enrich.py:enrich_sectors` sur toutes les années concaténées. Mutualiser ainsi le dictionnaire de tickers fait qu'un symbole vu une année sert aux autres.

La piste papier fournit **1 680** lignes, très concentrées sur Richard Blumenthal (environ 73 %). Ce papier est massivement composé d'**obligations municipales** (*munis*), non cotées : c'est lui qui explique la baisse de couverture ticker et secteur du Sénat en 2025–2026 (§5) — un effet de composition des actifs, non d'extraction.

Dans la source. Le papier du Sénat est un formulaire scanné (cf. annexe 11.1) : ex. « Microsoft, NASDAQ/OTC · 2/17/10 », fourchette de montant en colonnes cochées — comme la Chambre OCR, mais droit et sans symbole boursier imprimé (à ré-associer).

4.7 Donner un secteur à chaque ticker : GICS → ETF SPDR

La stratégie visée ne réplique pas les transactions action par action, mais **secteur par secteur**, au moyen d'ETF — des fonds cotés en Bourse qui répliquent tout un panier de titres, ici un secteur entier. Il faut donc, pour chaque ticker déclaré, déterminer son secteur. La référence est la classification **GICS**

(*Global Industry Classification Standard*), qui range chaque société cotée dans l'un de onze secteurs ; chacun est associé un-à-un à l'ETF correspondant de la famille *Select Sector SPDR* :

Energy→XLE · Materials→XLB · Industrials→XLI · Consumer Discretionary→XLY · Consumer Staples→XLP · Health Care→XLV · Financials→XLF · Information Technology→XLK · Communication Services→XLC · Utilities→XLU · Real Estate→XLRE

`common/sector_enrich.py:enrich_sectors` résout le secteur par une cascade hybride qui **trace sa source** (`sector_source`) : d'abord un secteur factuel via `yfinance` ; à défaut, un LLM (`claude-sonnet-4-6`) contraint de répondre dans l'énumération GICS stricte et de poser un drapeau `is_listed` — garde-fou qui interdit d'attribuer un secteur à un actif **non coté** ; enfin des overrides manuels issus d'un audit (les ETF de marché large comme `SPY` sont ainsi ramenés à *aucun* secteur). La table `GICS_TO ETF` fait la dernière marche. Le ticker lui-même est résolu en trois voies essayées en cascade : un symbole **explicite** déjà imprimé, un **dictionnaire** reliant un nom à un symbole vu ailleurs dans le corpus, puis un **LLM** muni d'un filtre « action » — pour ne jamais tickeriser une obligation ou une muni.

Une précision s'impose sur le mot **couverture** : c'est un *taux de remplissage*, non un taux d'exactitude. Les actifs non cotés — obligations, *munis*, fonds privés — n'ont, par nature, ni symbole ni secteur à recevoir ; une couverture inférieure à 100 % reflète donc avant tout la part d'actifs non cotés, et non un défaut d'extraction.

Au terme de l'enrichissement, le secteur est renseigné pour **83,2 %** des lignes à la Chambre et **62,1 %** au Sénat, le ticker pour **85,3 %** et **71,4 %**. Les montants sont ramenés au milieu de leur fourchette (`house/amounts.py:amount_midpoint`), et l'ancienneté (`years_in_office`) couvre **100 %** des lignes des deux chambres — calculée hors ligne et *appendue* en dernière étape par `common/enrich_tenure.py`, de façon octet-préservante.

4.8 La table finale : le contrat « 12/12 »

Ce qui assemble les quatre pistes sans rien perdre tient en un module, `common/schema.py`, et en une **clé naturelle** de sept champs : `chamber`, `declarant_name`, `transaction_date`, `asset_description`, `operation_type`, `amount_range` et `owner`. Cette clé exclut délibérément deux informations : le `ticker`, ajouté plus tard à l'enrichissement, dont la présence la rendrait instable ; et la `disclosure_date` (§2), car un dépôt initial et son amendement portent le même trade et doivent rester *une* transaction, non deux. Dédupliquer sur cette seule clé écraserait pourtant des lots réels — un déposant peut légitimement déclarer le même titre plusieurs fois dans un même dépôt, via plusieurs comptes. Pour les préserver, `add_occurrence_index` numérote ces répétitions *au sein d'un dépôt* (`occurrence_index`), et `dedup_canonical` ne supprime alors que les vrais doublons d'un dépôt à l'autre : la déduplication est **non destructrice**, opérée sur le couple (`natural_key_hash`, `occurrence_index`).

La table finale compte **28 colonnes** — vingt-sept d'assemblage, plus `years_in_office` appendue en dernier. Les **douze champs « métier »** exigés sont exactement la constante `schema.CORE_FIELDS` :

```
declarant_name · chamber · party · committee_membership · transaction_date · disclosure_date  
· ticker · sector_gics · etf_proxy · operation_type · amount_midpoint · asset_type
```

Reste une nuance d'honnêteté : « présents » n'est pas « sans valeurs manquantes ». Les douze champs figurent au schéma des deux chambres, mais trois comportent des trous **légitimes**, par nature et non par défaut d'extraction — `ticker`, `sector_gics` et `etf_proxy` sont vides pour les actifs non cotés, et `committee_membership` est un instantané *courant*, non l'historique daté des commissions. Les champs structurellement toujours renseignables — nom, chambre, parti, dates, opération, montant — sont, eux, quasi complets. Les taux de remplissage exacts, champ par champ et année par année, figurent en annexe 11.4.

5 Le rapport de qualité

Une table « pleine » n'est pas pour autant une table *saine* : ses dates peuvent être incohérentes, les délais légaux non respectés, son activité concentrée sur quelques déposants. `common/quality.py` (`build_report`) y répond par **six contrôles** et quatre figures, calculés en **lecture seule** des tables finales, sans aucun appel d'API (matplotlib en mode `Agg`) :

- (a) **Cohérence des dates** (`disclosure_date ≥ transaction_date`) : 99,8 % à la Chambre, 100,0 % au Sénat.
- (b) **Délai légal** : 86,9 % (Chambre) et 84,9 % (Sénat) des dépôts dans les 45 jours prévus par le STOCK Act, pour un délai médian de **28 jours**.
- (c) **Distribution des montants** : médiane à **8 000 \$**, quartile supérieur à **32 500 \$**.
- (d) **Couverture par membre** : 320 déposants, dont **206** ayant déclaré au moins dix transactions et **150** actifs sur au moins trois années.
- (e) **Achats sans sortie déclarée** : 22,5 % à la Chambre, 39,6 % au Sénat.
- (f) **Validation externe contre Quiver**, par scope — développée à la section suivante (§6), agrégée ici depuis les réconciliations figées (07c/07g/07h).

Le détail par an figure en annexe 11.4.

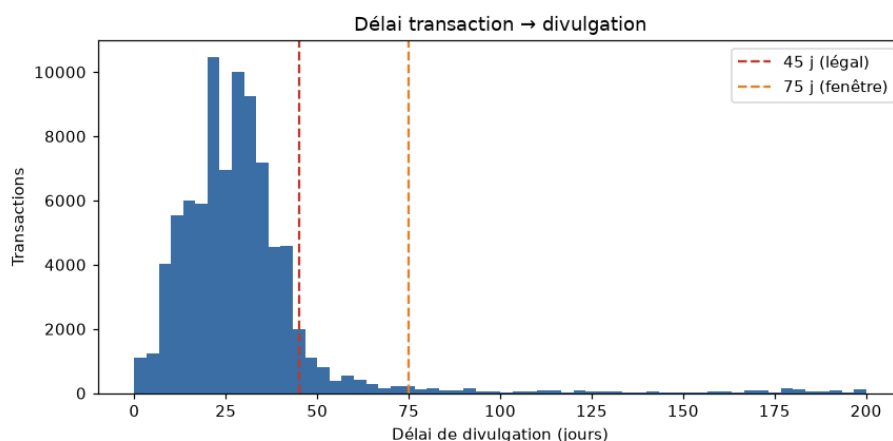


FIGURE 1 – Délai disclosure – transaction (0–200 j), lignes 45 j (légal) / 75 j (fenêtre). Générée par `common/quality.py` — lecture seule, aucun appel API.

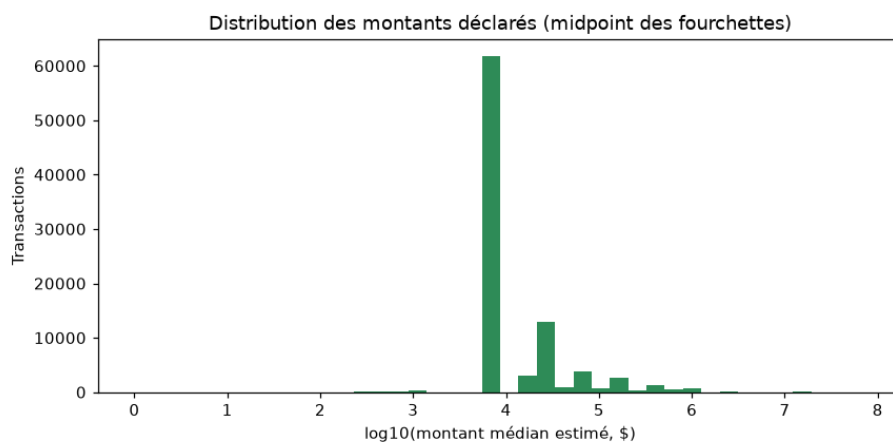


FIGURE 2 – Distribution de `amount_midpoint` en échelle `log10`. Générée par `common/quality.py`.

Le bilan global masque toutefois une dynamique. Au Sénat, la couverture en ticker chute à **45,4 %** en 2025 et **43,7 %** en 2026, contre environ 80 % les années précédentes, le secteur suivant la même pente. La cause n'est pas une dégradation de l'extraction mais une **composition d'actifs** : la part

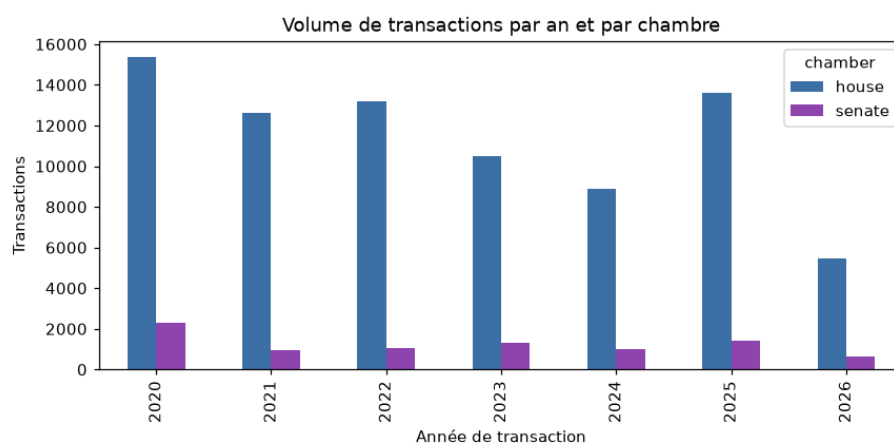


FIGURE 3 – Transactions par **année de transaction** et par chambre. Générée par `common/quality.py`.

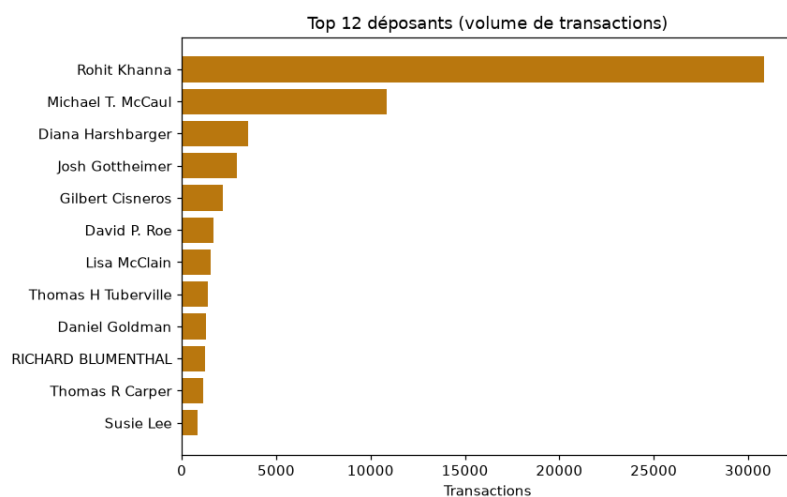


FIGURE 4 – Top 12 déposants par **nombre de transactions**. Générée par `common/quality.py` — la domination des déposants-papier (Khanna, McCaul) y est visible.

d'obligations municipales — non cotées, donc légitimement sans ticker — y augmente fortement. La fiabilité des dates issues de l'OCR est, elle, suivie par la colonne `date_confidence` : 95,3 % de dates jugées plausibles à la Chambre, le creux de 2021 (79,6 %) tenant entièrement aux deux documents manuscrits denses de Diana Harshbarger — une anomalie localisée, non systémique.

Cette colonne repose sur une fenêtre de plausibilité de **75 jours** : le STOCK Act vise environ 45 jours, et l'on tolère cette marge pour absorber le bruit de lecture de l'OCR. C'est une heuristique appliquée aux seules lignes OCR, non une vérité-terrain — ce rôle revient à Quiver, objet de la section suivante.

6 La validation externe (Quiver)

Vérifier la justesse de l'extraction suppose une source indépendante, sans se fier à sa propre sortie. On confronte donc la table finale à Quiver au niveau transaction — clé `bioguide` × `ticker` × `date` × `sens`, la date appariée étant le *Traded* de Quiver — **sans jamais le réinjecter**. L'apport central est de mener cette comparaison **par scope** (`common/quiver_scopes.py:reconcile_scopes`) : l'électronique seul, l'OCR seul, puis les deux ensemble. La *couverture* mesure la part des trades Quiver que l'on retrouve ; `only_ours` désigne nos trades absents de Quiver, `only_quiver` l'inverse. Une seconde fonction, `match_breakdown`, classe chaque transaction selon quatre cas — `exact_match` (même trade, même date), `date_mismatch` (bon trade, date divergente), `no_match` (absent de Quiver) et `non_equity` (muni ou obligation, hors périmètre Quiver) — ventilés par type d'actif et par cluster de scan.

Couverture par scope (transaction-niveau, toutes années) :

Scope	Chambre	Sénat	lecture
électronique (<code>digital</code>)	98,2 %	99,4 %	le parsing déterministe est quasi exact
OCR seul (<code>ocr</code>)	68,3 %	<i>vide</i>	cf. les deux constats ci-dessous
les deux (<code>both</code>)	86,2 %	99,4 %	union dédoublée des trades

Premier constat : Quiver ne suit que les actions cotées. La décomposition par type d'actif le montre : sur les actions, Quiver possède le trade dans 87,5 % des cas à la Chambre et 84,2 % au Sénat ; en revanche, les `non_equity` — obligations d'État, *mutual funds* et surtout obligations municipales — sont hors de son périmètre. Un `non_equity` non apparié n'est ni une erreur de notre part ni un défaut : c'est un actif que Quiver ne suit tout simplement pas. C'est là toute l'explication du Sénat, dont l'OCR papier est massivement non coté (les *munis* de Blumenthal) : le scope OCR y ressort « vide » côté Quiver, ces actifs lui échappant par nature. Les actions du Sénat, elles, sont très bien couvertes.

Second constat : l'OCR de la Chambre est validé en externe. Quiver référence aussi les gros déposants-papier de la Chambre, comme Khanna ou McCaul ; l'OCR de la Chambre est donc *validable* contre une source externe, et la décomposition par cluster de scan situe précisément la limite :

Cluster (Chambre)	Quiver a le trade	date juste	lecture
A — tapé droit	88,0 %	85,7 %	lecture fiable
B — tapé couché	77,9 %	68,2 %	le skew suffit, dates correctes
C — manuscrit	35,3 %	37,2 %	<i>point dur</i> → cluster exclu

La colonne « date juste » donne la part d'`exact_match` parmi les seuls trades que Quiver possède. La leçon est nette : même lorsque Quiver a le trade, c'est notre **lecture OCR de la date** qui décroche sur le manuscrit (37 %) — d'où l'exclusion du cluster C (§4.5).

Au bilan, aucune erreur d'extraction systématique n'apparaît. Sur l'électronique, la concordance est quasi parfaite (98–99 %) ; sur l'OCR, les écarts s'expliquent intégralement par le non-coté hors périmètre de Quiver et par la date des scans manuscrits. `only_ours` — nos trades absents de Quiver — est donc surtout du non-coté et du manuscrit, non du bruit. `common/crosscheck.py` matérialise ce statut déposant par déposant (`quiver_validable`, `ocr_unique`, `digital`), détaillé en annexe 11.4.

7 Assurance qualité & reproductibilité

Le pipeline n'est pas rejouable hors ligne : le *scraping* exige le réseau, l'OCR une API, et rejouer une année électronique produirait même une table vide, puisque les PDF lisibles ont été parsés puis écartés. La correction ne peut donc se prouver par simple ré-exécution ; on l'établit, plus solidement, par **reproduction déterministe depuis les colonnes figées**.

Quatre garde-fous s'y emploient. D'abord, un **golden octet-à-octet** : l'empreinte SHA-256 de chacune des sorties est figée — 125 fichiers à la Chambre, 76 au Sénat — et `check_golden.py` (respectivement `senate_check_golden.py`) doit afficher « ZÉRO ÉCART » à chaque exécution. Ensuite, des **reproductions fonction par fonction** : `test_senate_repro` recompose chaque hash de clé naturelle (8841/8841) et réassocie identité et ticker à partir des seules colonnes figées ; côté Chambre, `test_schema`, `test_amounts_tickers`, `test_identity` et `test_tenure` jouent le même rôle. Troisièmement, `test_vision_sha` et `test_incremental` vérifient que le `prompt_sha` est stable — un second passage OCR ne déclenche aucun appel. Enfin, `audit_metrics.py` recompute l'ensemble des métriques et *asserte* les invariants porteurs (totaux, identité) : c'est la source de vérité chiffrée de ce rapport. Les réconciliations Quiver elles-mêmes, déterministes grâce à un tri à départage stable, sont figées dans le golden au même titre que les tables.

8 Limites (honnêteté)

- **Cluster C manuscrit exclu par défaut** : la limite n'est pas la couverture Quiver (qui a le trade) mais notre **lecture OCR des dates manuscrites** (date juste 37 %) — choix assumé.
- **ticker/sector_gics/etf_proxy structurellement absents** pour munis et obligations (actifs non cotés) — par nature, pas par défaut. C'est aussi ce qui rend une partie de l'OCR **hors périmètre Quiver** (qui ne suit que les actions).
- **Commissions non « point-in-time »** : `committee_membership` est un *snapshot* courant, pas l'historique daté.
- **date_confidence = heuristique binaire** (fenêtre 75 j), **colonne OCR seulement** ; ce n'est pas une vérité-terrain (Quiver l'est).
- **Baisse de couverture Sénat 2025–26** : réelle mais *de composition* (montée des munis), pas d'extraction.

9 Périmètre & suite

La **couche données** décrite ici est livrée : 90 487 transactions extraites, identifiées, enrichies (table 12/12), validées et reproductibles. La **stratégie et le backtest** associés (copy-trading actions puis ETF sectoriels) constituent un travail de recherche séparé, hors-périmètre de ce rapport.

10 Conclusion

Sur deux chambres et sept ans, le pipeline reconstruit **90 487** transactions à partir des sources officielles, en traitant honnêtement les deux pistes (électronique déterministe *et* scannée par OCR Vision). Chaque ligne est identifiée (99,99 / 100 %), enrichie (table 12/12 sur les deux chambres) et validée contre Quiver sans jamais le réinjecter, **par scope** : 98,2 % sur l'électronique Chambre, 68,3 % sur l'OCR, 86,2 % sur l'ensemble ; 99,4 % au Sénat. Cette validation par scope localise notre seule vraie limite, la **lecture des dates manuscrites**, et montre que les écarts du Sénat viennent du **non-coté hors périmètre Quiver**, non d'erreurs. La correction est figée (golden octet-à-octet, 125/76) et reproductible. Surtout, le rapport assume ses nuances — « présents » n'est pas « sans manquants », la couverture Sénat 2025–26 baisse pour des raisons de composition d'actifs. C'est une couche données sur laquelle une stratégie peut être bâtie *en connaissant ses limites*.

Module	Rôle	Fonctions clés
schema.py	clé naturelle + dédup	natural_key_hash (SHA-256 de 7 champs) · add_occurrence_index · dedup_canonical (non destructrice) · CORE_FIELDS
sector_enrich.py	secteur GICS → ETF	resolve_yfinance · resolve_llm (is_listed) · enrich_sectors (+ sector_source) · GICS_TO ETF
vision_ocr.py	OCR Vision de référence	detect_rotation (deskew 4 rotations) · pdf_to_b64_images · VisionExtractor.prompt_sha · .extract_cached
crosscheck.py	statut par déposant	per_filer_status · add_external_counts (Kadoda/HSW) · summary
quality.py	rapport qualité (6 contrôles)	load_final · date_coherence · delay_buckets · amount_distribution · coverage_per_member · unmatched_purchase_rate · quiver_validation · build_report
enrich_tenure.py	ancienneté	add_years_in_office (append years_in_office octet-préservant) · main
pipeline.py	orchestrateur	build_steps (séquence 6 étapes) · main (sous-processus, -dry-run)
quiver_scopes.py	validation Quiver multi-scopes	reconcile_scopes (digital/ocr/both → 07c-07f) · match_breakdown (par actif & cluster → 07g/07h)
house/ — orchestrateur Chambre (7)		
digital.py	PTR électroniques	load_ptr_index (XML, FilingType=P) · resolve_pdf_path · build_manifest (tri lisible/non) · parse_ptr (TXN_RE, _join_split_lines) · finalize · validate_quiver · run_year
ocr.py	scans → FINAL (par an)	run_ocr_year (Vision + fusion + ticker + secteur + Quiver 07c-07h) · _infer_asset_type
identity.py	matcher Chambre	make_matcher (override→exact+surnoms→nom unique) · _MANUAL_BIO
amounts.py	montant & opération	amount_midpoint (milieu de fourchette) · operation_type_from_code (P/S/E) · HOUSE_OCR_AMOUNT_MAP
tickers.py	ticker + type d'actif	explicit_ticker · recover_ticker · infer_asset_type (par chambre)
quiver.py	réconciliation Quiver	reconcile · norm_ticker (robuste NaN/vide)
echantillon.py	pilote OCR stratifié	run_echantillon · compare_quiver
senate/ — orchestrateur Sénat (8)		
digital.py	eFD électronique	accept_agreement (CSRF) · fetch_ptr_list (report_types=[11]) · fetch_report · parse_electronic (pd.read_html, _find_col) · run_year
ocr.py	papier .gif → 06b	discover_index · ocr_one (un .gif → Vision) · build_table
ocr_engine.py	moteur OCR figé	extract_report · normalize · _infer_asset_type
fusion.py	fusion + enrich corpus	main (Étape 1 fusion/an · Étape 2 enrichissement corpus + Quiver 07c-07g) · date_confidence
identity.py	matcher Sénat	load_reference · make_matcher (désambiguïsation chambre, exclut délégués) · enrich
ticker.py	ticker 3 voies	resolve_tickers (explicit→dict→LLM) · llm_resolve_tickers
quiver.py	réconciliation Quiver	reconcile · norm_ticker · norm_sense
census_probe.py	Phase 0 (census)	parse_electronic · main (census + sondage parser)

Les `__init__.py` (exports de paquet) et `tests/regression/` (golden + reproductions) complètent le dépôt. Le secteur (`common/sector_enrich`) et la clé (`common/schema`) sont importés en direct par les deux orchestrateurs — d'où leur place dans `common/`.

11.3 Annexe C — le schéma FINAL (28 colonnes)

#	Colonne	Sens	Remplie par
1	bioguide_id	identifiant officiel du membre	identity
2	declarant_name	nom tel que déposé	parse / OCR
3	chamber	house / senate	enrich_identity
4	party	parti	Reference.party
5	state_district	état + district	03_ptr_index
6	committee_membership	commissions (liste)	Reference.committees
7	committees_key_flag	commission « clé » ?	reference
8	transaction_date	date de la transaction	parse / OCR
9	disclosure_date	date de divulgation (dépôt)	03_ptr_index / eFD
10	ticker	symbole boursier	tickers / ticker
11	asset_description	libellé de l'actif	parse / OCR
12	asset_type	type d'actif	infer_asset_type
13	sector_gics	secteur GICS	sector_enrich
14	etf_proxy	ETF SPDR du secteur	GICS_TO_ETF
15	operation_type	Purchase / Sale / ...	operation_type_from_code
16	amount_range	fourchette déclarée	parse / OCR
17	amount_midpoint	milieu de la fourchette	amount_midpoint
18	amount_split_flag	lot multi-actifs ?	finalize
19	owner	propriétaire (self/spouse...)	normalize
20	doc_id	identifiant du PTR	03_ptr_index
21	source_url	URL de la source	03_ptr_index
22	natural_key_hash	clé de dédup (7 champs)	schema
23	occurrence_index	rang du lot intra-dépôt	add_occurrence_index
24	provenance	*-electronic / *-ocr	pipeline
25	date_confidence	plausible / implausible (75 j)	date_confidence
26	ticker_source	explicit / elec_dict / llm / none	tickers / ticker
27	sector_source	yfinance / llm / manual / none	sector_enrich
28	years_in_office	ancienneté à la date du trade	enrich_tenure

L'ordre exact diffère légèrement entre Chambre et Sénat (date_confidence plus tôt au Sénat); le contenu garanti — les 12 champs « métier » — est identique. Les 27 colonnes d'assemblage (HOUSE_FINAL_SCHEMA / SENATE_FINAL_SCHEMA) + years_in_office = 28.

11.4 Annexe D — métriques vérifiées (recalculées depuis les FINAL)

Lue après le rapport, cette annexe dit si l'extraction a bien marché. Tous les chiffres sont recomputés depuis les tables FINAL par tests/regression/audit_metrics.py, common/quality.py et les réconciliations Quiver figées (07c-07h).

D.1 — Volumes par an.

An	Chambre				Sénat			
	élec.	OCR	FINAL	dépos.	élec.	OCR	FINAL	sén.
2020	6 886	8 791	15 677	109	1 706	255	1 961	33
2021	5 457	6 816	12 273	120	1 098	281	1 379	35
2022	3 601	10 900	14 501	110	919	182	1 101	27
2023	4 161	6 329	10 490	100	1 062	97	1 159	30
2024	2 694	6 277	8 971	97	946	84	1 030	24
2025	7 577	6 067	13 644	107	943	478	1 421	31
2026	2 300	3 790	6 090	84	487	303	790	22
Tot.	32 676	48 970	81 646	256*	7 161	1 680	8 841	64*

* bioguides *uniques* sur 7 ans (la somme annuelle compte un élu chaque année).

D.2 — Couverture des champs enrichis, par an (% renseigné).

An	Chambre				Sénat			
	ticker	sect.	a.type	comm.	ticker	sect.	a.type	comm.
2020	86,5	83,4	91,0	64,9	88,1	80,5	96,3	34,5
2021	87,3	85,8	90,1	78,7	80,9	70,1	94,2	54,3
2022	87,1	85,2	91,0	89,2	79,7	65,3	96,1	78,3
2023	78,9	76,6	88,6	96,8	77,7	69,5	96,3	71,4
2024	81,3	78,3	87,4	93,6	68,0	59,2	100	77,8
2025	87,8	86,3	94,8	97,6	45,4	37,2	100	84,9
2026	85,4	84,4	94,9	99,7	43,7	35,7	100	86,2
Glob.	85,3	83,2	91,1	86,6	71,4	62,1	97,3	65,6

D.3 — Les 6 contrôles qualité (global).

Contrôle	Chambre	Sénat
(a) dates cohérentes (<code>disclosure ≥ transaction</code>)	99,8 %	100,0 %
(b) dépôts ≤ 45 j (délai médian 28 j)	86,9 %	84,9 %
(c) montants — médiane / quartile haut	8 000 \$ / 32 500 \$	
(d) déposants (≥ 10 trades / ≥ 3 ans)	320 (206 / 150)	
(e) achats sans sortie déclarée	22,5 %	39,6 %
(f) validation Quiver — scope « les deux »	86,2 %	99,4 %

D.4 — Validation Quiver par scope, par an (couverture % = trades Quiver retrouvés; `common/quiver_scopes.py`).

An	Chambre			Sénat		
	élec.	OCR	les deux	élec.	OCR	les deux
2020	99,7	61,3	85,1	99,4	—	99,4
2021	99,6	56,0	88,6	98,0	—	98,0
2022	98,2	68,5	80,9	99,4	—	99,4
2023	96,0	67,1	81,0	99,7	0,0	99,7
2024	92,0	78,8	85,8	99,6	0,0	99,6
2025	99,2	61,2	91,3	99,8	—	99,8
2026	98,6	85,3	90,6	100,0	—	100,0
Glob.	98,2	68,3	86,2	99,4	0,0	99,4

Le scope OCR du Sénat est « — » (Quiver ne possède aucun de ces trades : ce sont surtout des munis non cotées). Le scope « les deux » est l'*union dédoublée*, pas la somme.

D.5 — Décomposition Quiver par type d'actif et par cluster (toutes années ; 07g/07h).

Chambre — type d'actif	exact	date ≠	no_match	non_equity	Quiver a %
Stock	47 475	9 301	8 105	2 796	87,5
Mutual Fund	130	92	715	575	23,7
Gov Security	5	0	23	2 259	17,9
Cluster de scan	exact	date ≠	no_match	non_equity	date juste %
A — tapé droit	3 784	630	600	943	85,7
B — tapé couché	18 558	8 653	7 706	7 234	68,2
C — manuscrit	100	169	493	100	37,2

$Quiver\ a\ \% = (exact + date \neq) / actions$; $date\ juste\ \% = exact / (exact + date \neq)$. Sénat : les **Stock** sont couverts à 84,2 %; l'OCR y est surtout du non-coté (Municipal Security, Corporate Bond) → hors périmètre Quiver.

D.6 — Fiabilité des dates OCR (date_confidence, lignes OCR seulement).

An	Chambre (OCR)		Sénat (OCR)	
	plausible %	implausibles	plausible %	implausibles
2020	98,6	120	96,1	9
2021	79,6	1 389	97,5	6
2022	96,4	394	99,5	0
2023	99,2	50	100,0	0
2024	96,5	220	97,6	0
2025	98,1	118	99,0	3
2026	100,0	0	99,7	0
Tot.	95,3	2 291	98,5	18

Le creux 2021 Chambre est entièrement Diana Harshbarger (2 documents manuscrits denses).

D.7 — Clusters OCR & concentration.

Clusters Chambre (census 547 scans)	Concentration des déposants OCR
A — tapé droit : 74 docs	Chambre : Khanna 30 862 (62,9 %), McCaul 10 876 (22,2 %),
B — tapé couché : 322 docs	Harshbarger 3 514 (7,2 %) → top 3 = 92,4 %, top 10 = 99,0 %
C — manuscrit : 151 docs (exclu)	Sénat : Blumenthal 1 233 (73 %), Boozman 379, Burr 52, Feinstein 14, Fetterman 2

D.8 — Statut de validation par déposant (Chambre, crosscheck).

Statut	dépôts	transactions	dont OCR
quiver_validable (Quiver le couvre)	212	78 530	47 004
ocr_unique (Quiver = 0, OCR ≥ 50 %)	13	1 957	1 949
digital (peu / pas d'OCR)	31	1 155	13

Identité : Chambre 256 bioguides / 275 noms (99,99 %), Sénat 64 / 67 (100 %). Golden (zéro écart) : **125** fichiers Chambre, **76** Sénat. **Total : 90 487 transactions**, 2020–2026.

11.5 Annexe E — reproduire / lancer

Installation

```
python3.12 -m venv .venv && ./venv/bin/pip install -e .
```

Pipeline complet (reseau + API Vision requis) ; --dry-run = voir la sequence

```
./venv/bin/python -m common.pipeline --years 2020-2026
```

```
./venv/bin/python -m common.pipeline --years 2024 --dry-run
```

Rapport de qualite (offline, lecture seule des FINAL)

-> docs/RAPPORT_QUALITE.md + docs/quality/*.png

```
./venv/bin/python -m common.quality
```

Filet "zero changement" (doit afficher ZERO ECART)

```
./venv/bin/python tests/regression/check_golden.py # Chambre, 125
```

```
./venv/bin/python tests/regression/senate_check_golden.py # Senat, 76
```

```
./venv/bin/python tests/regression/test_senate_repro.py # reproductions
```

Document dérivé et vérifié sur le code source réel et les tables FINAL du dépôt. Toute divergence avec une autre documentation doit être tranchée en faveur du code et des données. Quiver = vérification externe, jamais réinjecté.